



Chapter 8 - Cluster Analysis - Basic Concepts and Methods

J. Han, J. Pei and H. Tong, *Data Mining, Concepts and Techniques*, 4th ed.,
Morgan Kaufmann, 2023.



Contents

- 1. Cluster Analysis
- 2. Partition Methods
- 3. Evaluation of Clustering



1. Cluster Analysis

- *Clustering* is the process of grouping a set of data objects into multiple groups or *clusters* so that objects within a cluster have high similarity, but are dissimilar to objects in other clusters.
- Dissimilarities and similarities are assessed based on the attribute values describing the objects and often involve distance measures.
- Clustering as a data mining tool has its roots in many application areas, such as biology, security, business intelligence, and Web search.



1. Cluster Analysis

1.1. What is Cluster Analysis?

- **Cluster analysis** or simply **clustering** is the process of partitioning a set of data objects (or observations) into subsets.
 - Each subset is a **cluster**, such that objects in a cluster are similar to one another, yet dissimilar to objects in other clusters.
- The set of clusters resulting from a cluster analysis can be referred to as a **clustering**.
 - In this context, different clustering methods may generate different clusterings on the same data set. The same clustering method equipped with different parameters or even different initializations may also produce different clusterings.
- Such partitioning is not performed by humans, but by a clustering algorithm. Hence, clustering is useful in that it can lead to the discovery of previously unknown groups within the data.



1. Cluster Analysis

1.1. What is Cluster Analysis?

- As a data mining function, cluster analysis can be used as a standalone tool to gain insight into the distribution of data, to observe the characteristics of each cluster, and to focus on a particular set of clusters for further analysis.
- Alternatively, it may serve as a preprocessing step for other algorithms, such as characterization, attribute subset selection, and classification, which would then operate on the detected clusters and the selected attributes or features.
- Because a cluster is a collection of data objects that are similar to one another within the cluster and dissimilar to objects in other clusters, **a cluster of data objects can be treated as an implicit class**. In this sense, clustering is sometimes called **automatic classification** or **unsupervised classification**. Again, a critical difference here is that clustering can automatically find the groupings. This is a distinct advantage of cluster analysis.



1. Cluster Analysis

1.1. What is Cluster Analysis?

- Clustering is also called **data segmentation** in some applications because clustering partitions large data sets into groups according to their *similarity*.
 - Clustering can also be used for **outlier detection**, where outliers (values that are “far away” from any cluster) may be more interesting than common cases.



1. Cluster Analysis

1.2. Requirements for Cluster Analysis

- When we think about employing a clustering method, what requirements should we consider for the method? The following are typical requirements of clustering in data mining.
- **Ability to deal with various kinds of data objects:**
 - Many algorithms are designed to cluster numeric (interval-based) data objects.
- **Scalability:**
 - Many clustering algorithms work well on small data sets containing fewer than several hundred data objects. Clustering on only a sample of a given large data set may lead to biased results. Therefore highly scalable clustering algorithms are needed.



1. Cluster Analysis

1.2. Requirements for Cluster Analysis

- **Discovery of clusters with arbitrary shape:**
 - Many clustering algorithms determine clusters based on Euclidean or Manhattan distance measures . Algorithms based on such distance measures tend to find spherical clusters with similar size and density. However, a cluster could be of any shape.
 - For example, we may want to use clustering to find the frontier of a running forest fire in a satellite image, which is often not spherical.
 - It is important to develop algorithms that can detect clusters of arbitrary shape.



1. Cluster Analysis

1.2. Requirements for Cluster Analysis

- **Requirements for domain knowledge to determine input parameters:**
 - Many clustering algorithms require users to provide domain knowledge in the form of input parameters such as the desired number of clusters.
 - Consequently, the clustering results may be sensitive to such parameters. Parameters are often hard to determine, especially for data sets of high dimensionality where users have yet to grasp a deep understanding of their data. Requiring the specification of domain knowledge not only burdens users, but also makes the quality of clustering difficult to control.
 - Clustering algorithms that do not heavily rely on domain knowledge input or can help users explore domain knowledge are highly preferable.



1. Cluster Analysis

1.2. Requirements for Cluster Analysis

- **Ability to deal with noisy data:**
 - Most real-world data sets contain outliers and/or missing, unknown, or erroneous data. For example, data collected by physical sensors is often noisy. Clustering algorithms can be sensitive to such noise and may produce poor-quality clusters. Therefore, we need clustering methods that are robust to noise.
- **Incremental clustering and insensitivity to input order:**
 - In many applications, incremental updates (representing newer data) may arrive at any time. Some clustering algorithms cannot incorporate incremental updates into existing clustering structures and, instead, have to recompute a new clustering from scratch. Clustering algorithms may also be sensitive to the input data order. That is, given a set of data objects, clustering algorithms may return dramatically different clusterings depending on the order in which the objects are presented. Incremental clustering algorithms and algorithms that are insensitive to the input order are needed.



1. Cluster Analysis

1.2. Requirements for Cluster Analysis

- **Capability of clustering high-dimensionality data:**
 - A data set can contain numerous dimensions or attributes. Most clustering algorithms are good at handling low-dimensional data such as data sets involving only two or three dimensions. Finding clusters of data objects in a high-dimensional space is challenging, especially considering that such data can be very sparse and highly skewed.
- **Constraint-based clustering:**
 - Real-world applications may need to perform clustering under various kinds of constraints. Suppose that your task is to choose the locations for a given number of new electric vehicle charging stations in a city. To decide upon this, you may cluster potential charging needs while considering constraints such as available spaces, electricity networks, and the river and highway networks in a city. A challenging task is to find data groups with good clustering behaviors that satisfy specified constraints.



1. Cluster Analysis

1.2. Requirements for Cluster Analysis

- **Interpretability and usability:**
 - Users want clustering results to be interpretable, comprehensible, and usable. That is, clustering may need to be tied with specific semantic interpretations and applications. It is important to study how an application goal may influence the selection of clustering features and clustering methods.



2. Partitioning Methods

- The simplest and most fundamental version of cluster analysis is partitioning.
- In partitioning clustering,
 - We organize the objects in a given set into several exclusive groups or clusters.
 - Each cluster can be typified by a representative. In other words, each object o can be assigned to the cluster whose representative that o is the closest or most similar to.
- To keep the problem specification concise, we can assume that **the number of expected clusters is given**. This parameter is the starting point for partitioning methods.
- There are two foremost technical issues in partitioning methods.
 - First, how can we decide the representatives of clusters?
 - Second, how can we measure the distance or similarity between objects or between objects and representatives.



2. Partitioning Methods

- Formally, given a data set, D , of n objects, and k , the number of clusters to form, a partitioning algorithm organizes the objects into k partitions ($k \leq n$), where each partition represents a cluster.
- The clusters are formed **to optimize an objective partitioning criterion**, such as a dissimilarity function based on distance, so that the objects within a cluster are “similar” to one another and “dissimilar” to the objects in other clusters in terms of the data set attributes.



2. Partitioning Methods

2.1 k -Means: A Centroid-Based Technique

- Suppose a data set, D , contains n objects in Euclidean space.
- Partitioning methods distribute the objects in D into k clusters, $C_1, \dots, C_k$, that is, $C_i \subset D$, $|C_i| \geq 1$, and $C_i \cap C_j = \emptyset$ for $(1 \leq i, j \leq k, i \neq j)$.
 - Each cluster is required to have at least one object.
 - An objective function is used to assess the partitioning quality so that objects within a cluster are similar to one another but dissimilar to objects in other clusters. This is, the objective function aims for high intracluster similarity and low intercluster similarity.



2. Partitioning Methods

2.1 k -Means: A Centroid-Based Technique

- A centroid-based partitioning technique uses the *centroid* of a cluster, $\mathbf{c}_i$, as the representative of that cluster. Conceptually, the centroid of a cluster is its center point. The centroid can be defined in various ways such as
 - The mean
 - The medoid (A medoid is a representative point within a cluster that minimizes the average dissimilarity (or distance) to all other points in the same cluster. Unlike a mean, a medoid is an actual data point.)
- The difference between an object $\mathbf{p} \in C_i$ and $\mathbf{c}_i$, the representative of the cluster, is measured by $\text{dist}(\mathbf{p}, \mathbf{c}_i)$, where $\text{dist}(x, y)$ is the Euclidean distance between two points x and y .
- The quality of cluster C_i can be measured by the **within-cluster variation**, which is the sum of *squared error* between all objects in C_i and the centroid $\mathbf{c}_i$, defined as $\sum_{\mathbf{p} \in C_i} \text{dist}(\mathbf{p}, \mathbf{c}_i)^2$
- The objective function tries to make the resulting k clusters as compact and separate as possible. The task of partitioning clustering can be modeled as to minimize the within-cluster variation E among all possible assignments of objects into clusters.

$$E = \sum_{i=1}^k \sum_{\mathbf{p} \in C_i} \text{dist}(\mathbf{p}, \mathbf{c}_i)^2$$



2. Partitioning Methods

2.1 k -Means: A Centroid-Based Technique

- Within-cluster variation

$$E = \sum_{i=1}^k \sum_{p \in C_i} \text{dist}(\mathbf{p}, \mathbf{c}_i)^2 \quad (8.1)$$

- Minimizing the within-cluster variation is computationally challenging. **In the worst case, we would have to enumerate the number of all possible partitionings.**
 - It is easy to see that the number of all possible partitionings is exponential to the number of objects. It has been shown that the problem is NP-hard in the general Euclidean space even for two clusters (i.e., $k = 2$).
 - To overcome the prohibitive computational cost for the exact solution, greedy approaches are often used in practice. A prime example is the k -means algorithm, which is simple and commonly used.



2. Partitioning Methods

2.1 k -Means: A Centroid-Based Technique

- “*How does the k -means algorithm work?*”
- The k -means algorithm defines the centroid of a cluster as the mean value of the points within the cluster. It proceeds as follows.
 - First, it randomly selects k objects from D , each of which initially represents a cluster mean or center.
 - For each of the remaining objects, an object is assigned to the cluster to which it is the most similar, based on the Euclidean distance between the object and the chosen means.
 - The k -means algorithm then iteratively improves the within-cluster variation. For each cluster, it computes the new mean using the objects assigned to the cluster in the previous iteration.
 - All the objects are then reassigned using the updated means as the new cluster centers. The iterations continue until the assignment is stable, that is, the clusters formed in the current round are the same as those formed in the previous round.



2. Partitioning Methods

2.1 k -Means: A Centroid-Based Technique

Algorithm: k -means. The k -means algorithm for partitioning, where each cluster's center is represented by the mean value of the objects in the cluster.

Input:

- k : the number of clusters,
- D : a data set containing n objects.

Output: A set of k clusters.

Method:

- (1) arbitrarily choose k objects from D as the initial cluster centers;
- (2) **repeat**
- (3) (re)assign each object to the cluster to which the object is the most similar;
- (4) update the cluster centers, that is, calculate the mean value of the objects for each cluster;
- (5) **until** no change;

FIGURE 8.1

The k -means partitioning algorithm.

2. Partitioning Methods

2.1 k -Means: A Centroid-Based Technique

- **Example 8.1. Clustering by k -means partitioning.** Consider a set of objects located in 2-D space, as depicted in Fig. 8.2(a). Let $k = 3$, that is, the user would like to partition the objects into three clusters.

+

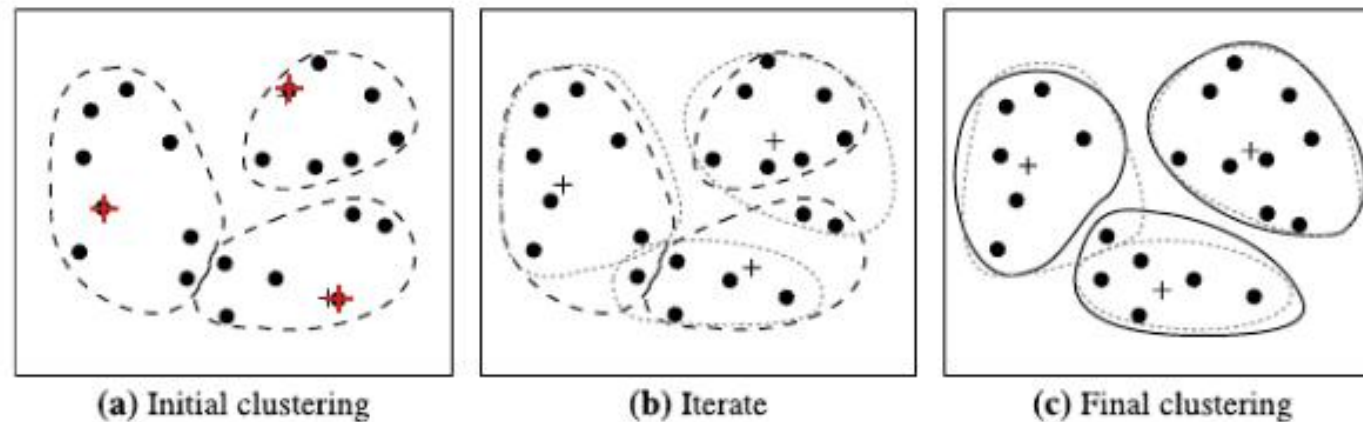


FIGURE 8.2

Clustering of a set of objects using the k -means method; for (b) update cluster centers and reassign objects accordingly (the mean of each cluster is marked by a +).



2. Partitioning Methods

2.1 k -Means: A Centroid-Based Technique

- The k -means method is not guaranteed to converge to the global optimum and often terminates at a local optimum.
- The results may depend on the **initial random selection of cluster centers**.
 - To obtain good results in practice, it is common to run the k -means algorithm multiple times with different initial cluster centers. The clustering with the smallest within-cluster variation should be returned as the final result.
- The time complexity of the k -means algorithm is $O(nkt)$, where n is the total number of objects, k is the number of clusters, and t is the number of iterations. Normally, $k \ll n$ and $t \ll n$. Therefore the method is relatively scalable and efficient in processing large data sets.



2. Partitioning Methods

2.1 k -Means: A Centroid-Based Technique

- The k -means method has several advantages.
 - 1. The k -means method is conceptually intuitive and relatively simple to implement.
 - 2. The k -means method is scalable to large data sets.
 - 3. The k -means method is guaranteed to converge to some local optimum, and thus likely does not produce very poor results.
 - 4. If a user has some domain knowledge about the possible locations of the clusters, the user can set the initial means and then run the iteration steps. In other words, the k -means method can take a warm-start.
 - 5. The k -means method can take new observed data easily. That is, if some new data objects arrive after a certain number of iterations, the k -means method still can easily take those data into the next iteration, and the updated clustering can adapt to the new data.



2. Partitioning Methods

2.1 k -Means: A Centroid-Based Technique

- The k -means method also has some limitations.
 - 1. A user has to manually specify the number of clusters. When a user is not familiar with a data set, it is not easy to set this parameter properly.
 - 2. The effect of result clustering heavily depends on the choice of initial means. When the number of clusters is small, to overcome this limitation, one may run the k -means method multiple times with different initial means. However, when the number of clusters is large, even running the k -means method multiple times may not help to mitigate the issue, since it is unlikely all clusters produced in a run are all good.
 - 3. The k -means method may meet difficulty in finding clusters of substantially different sizes and density. Moreover, outliers may distract the centers of clusters.
 - 4. Since the Euclidean distance is used in the k -means method, when the dimensionality increases, the distance measure is mainly dominated by noise. In expectation, the distance between any two data objects in a high dimensional space is the same. Thus the k -means method cannot scale up to high dimensional data straightforwardly.

2. Partitioning Methods

2.2 Variations of k -Mean

k -Medoids: A Representative Object-Based Technique

- The k -means algorithm is sensitive to outliers because such objects are far away from the majority of the data, and thus, when assigned to a cluster, they can dramatically distort the mean value of the cluster. This inadvertently affects the assignment of other objects to clusters.
 - This effect is particularly exacerbated due to the use of the *squared-error* function.

2. Partitioning Methods

2.2 Variations of k -Mean

k -Medoids: A Representative Object-Based Technique

- “How can we modify the k -means algorithm to diminish such sensitivity to outliers?”
- The basic of k -medoid method:
 - Instead of taking the mean value of the objects in a cluster as a reference point, we can pick **actual objects to represent the clusters**, using one representative object per cluster.
 - Each remaining object is assigned to the cluster of which the representative object is the most similar.
 - An **absolute-error criterion** is used, defined as

$$E = \sum_{i=1}^k \sum_{p \in C_i} \text{dist}(\mathbf{p}, \mathbf{o}_i)$$

where E is the sum of the absolute error for all objects $\mathbf{p}$ in the data set, and $\mathbf{o}_i$ is the representative object of C_i .

2. Partitioning Methods

2.2 Variations of k -Mean

k -Medoids: A Representative Object-Based Technique

- **Partitioning Around Medoids (PAM)**
 - The **Partitioning Around Medoids (PAM)** algorithm is a popular realization of k -medoids clustering. It tackles the problem in an iterative, greedy way. Like the k -means algorithm, the initial representative objects (called seeds) are chosen arbitrarily.
 - We consider whether replacing a representative object by a nonrepresentative object would improve the clustering quality. All the possible replacements are tried out. The iterative process of replacing representative objects by other objects continues until the quality of the resulting clustering cannot be improved by any replacement.
 - This quality is measured by a cost function of the average dissimilarity between an object and the representative object of its cluster.

2. Partitioning Methods

2.2 Variations of k -Mean

k -Medoids: A Representative Object-Based Technique

Algorithm: k -medoids. PAM, a k -medoids algorithm for partitioning based on medoid or central objects.

Input:

- k : the number of clusters,
- D : a data set containing n objects.

Output: A set of k clusters.

Method:

- The cost function calculates the *difference* in absolute-error value if a current representative object is replaced by a nonrepresentative object.
 - The total cost of swapping is the sum of costs incurred by all nonrepresentative objects.
 - If the total cost is negative, then o_j is replaced or swapped with o_{random} because the actual absolute-error E is reduced.
- (1) arbitrarily choose k objects in D as the initial representative objects or seeds;
 - (2) **repeat**
 - (3) assign each remaining object to the cluster with the nearest representative object;
 - (4) randomly select a nonrepresentative object, o_{random} ;
 - (5) compute the total cost, S , of swapping representative object, o_j , with o_{random} ;
 - (6) **if** $S < 0$ **then** swap o_j with o_{random} to form the new set of k representative objects;
 - (7) **until** no change;

FIGURE 8.4

PAM, a k -medoids partitioning algorithm.

2. Partitioning Methods

2.2 Variations of k -Mean

k -Medoids: A Representative Object-Based Technique

- “*Which method is more robust— k -means or k -medoids?*”
- The k -medoids method is more robust than k -means in the presence of noise and outliers because a medoid is less influenced by outliers or other extreme values than a mean.
- However, the complexity of each iteration in the k -medoids algorithm is $O(k^2(n - k)^2)$. For large values of n and k , such computation becomes very costly and much more costly than the k -means method.
- Both methods require the user to specify k , the number of clusters.

2. Partitioning Methods

2.2 Variations of k -Mean

k -Modes: Clustering Nominal Data

- One limitation of the k -means method is that it can be applied only when the mean of a set of objects is defined.
 - This may not be the case in some applications such as when data with nominal attributes is involved.
- The k -modes method is a variant of k -means, which extends the k -means paradigm to cluster nominal data by replacing the means of clusters with modes.
 - The mode for a set of data is the value that occurs most frequently in the set.

2. Partitioning Methods

2.2 Variations of k -Means

k -Modes: Clustering Nominal Data

- In order to use modes in clustering, we need a new way to compute the distance between two objects.
- Given two objects $\mathbf{x} = (x_1, \dots, x_l)$ and $\mathbf{y} = (y_1, \dots, y_l)$, we define the distance

$$\text{dist}(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^l d(x_i, y_i) \quad (8.3)$$

where $d(x, y) = 1$ if $x \neq y$ and otherwise 0.

- The k -modes method works largely the same way as the k -means method.
 - First, it selects k initial modes, one for each cluster.
 - Second, it allocates an object to the cluster whose mode is the closest to the object using the distance function in Eq. (8.3).
 - Third, it updates the mode of each cluster. For a cluster i and dimension j , the mode is updated to **the most frequent value on the dimension of all objects assigned to this cluster**. If there are more than one such a value, that is, if two or more values of the same frequency happen most frequently in the cluster, we can randomly choose one.
 - The k -modes method iterates the object allocation and mode update steps until either the sum of squared error (Eq. (8.1)) stabilizes or a given number of iterations are conducted.

2. Partitioning Methods

2.2 Variations of k -Means

k -Modes: Clustering Nominal Data

- <https://www.analyticsvidhya.com/blog/2021/06/kmodes-clustering-algorithm-for-categorical-data/>
 - Step 1: Pick $K = 3$ observations at random and use them as clusters: **P1, P7, P8**

person	hair color	eye color	skin color
P1	blonde	amber	fair
P2	brunette	gray	brown
P3	red	green	brown
P4	black	hazel	brown
P5	brunette	amber	fair
P6	black	gray	brown
P7	red	green	fair
P8	black	hazel	fair

Leaders			
P1	blonde	amber	fair
P7	red	green	fair
P8	black	hazel	fair
person	hair color	eye color	skin color
P1	blonde	amber	fair
P2	brunette	gray	brown
P3	red	green	brown
P4	black	hazel	brown
P5	brunette	amber	fair
P6	black	gray	brown
P7	red	green	fair
P8	black	hazel	fair

2. Partitioning Methods

2.2 Variations of k -Means

k -Modes: Clustering Nominal Data

- Step 2: Calculate the dissimilarities (no. of mismatches) and assign each observation to its closest cluster

Leaders			
P1	blonde	amber	fair
P7	red	green	fair
P8	black	hazel	fair
person	hair color	eye color	skin color
P1	blonde	amber	fair
P2	brunette	gray	brown
P3	red	green	brown
P4	black	hazel	brown
P5	brunette	amber	fair
P6	black	gray	brown
P7	red	green	fair
P8	black	hazel	fair

	Cluster 1 (P1)	Cluster 2 (P7)	Cluster 3 (P8)	Cluster
P1	0 ✓	2	2	Cluster 1
P2	3 ✓	3	3	Cluster 1
P3	3	1 ✓	3	Cluster 2
P4	3	3	1 ✓	Cluster 3
P5	1 ✓	2	2	Cluster 1
P6	3	3	2 ✓	Cluster 3
P7	2	0 ✓	2	Cluster 2
P8	2	2	0 ✓	Cluster 3

2. Partitioning Methods

2.2 Variations of k -Means

k -Modes: Clustering Nominal Data

- Step 3: Define new modes for the clusters.
 - Mode is simply the most observed value.

	Cluster 1 (P1)	Cluster 2 (P7)	Cluster 3 (P8)	Cluster	person	hair color	eye color	skin color
P1	0 ✓	2	2	Cluster 1	P1	blonde	amber	fair
P2	3 ✓	3	3	Cluster 1	P2	brunette	gray	brown
P3	3	1 ✓	3	Cluster 2	P3	red	green	brown
P4	3	3	1 ✓	Cluster 3	P4	black	hazel	brown
P5	1 ✓	2	2	Cluster 1	P5	brunette	amber	fair
P6	3	3	2 ✓	Cluster 3	P6	black	gray	brown
P7	2	0 ✓	2	Cluster 2	P7	red	green	fair
P8	2	2	0 ✓	Cluster 3	P8	black	hazel	fair

New Leaders			
	hair color	eye color	skin color
Cluster 1	brunette	amber	fair
Cluster 2	red	green	fair
Cluster 3	black	hazel	brown

2. Partitioning Methods

2.2 Variations of k -Means

k -Modes: Clustering Nominal Data

- The k -means and the k -modes methods can be integrated to cluster data with mixed numeric and nominal values. This is known as the **k -prototype** method.
- On each dimension, according to whether it is a numeric attribute or a nominal attribute, we can use either the absolute error $dist(x - y)$ or the mode difference $d(x, y) = 1$ if $x \neq y$ and otherwise 0.
- Since numeric attributes may have a much larger range in absolute error than the mode difference on nominal attributes, we can associate with each dimension a weight balancing the effect of each dimension.

2. Partitioning Methods

2.2 Variations of k -Means

Initialization in Partitioning Methods

- Choosing the initial cluster centers carefully, we may be able to not only speed up the convergence of the k -means algorithm, but also guarantee the quality of the final clustering results.
- For example, the **k -means++** algorithm chooses the initial centers in the following steps.
 - First, it chooses one center uniformly at random from the objects in the data set.
 - Iteratively, for each object p other than the chosen centers, it chooses the object as the new center at random with probability proportional to $D(p)^2$, where $D(p)$ is the distance from p to the closest center that has already been chosen. The iteration continues until k centers are chosen.
- Extensive experimental results have shown that the k -means++ algorithm can speed up the clustering process by a factor from 2 in most cases. Moreover, the k -means++ algorithm guarantees an approximation ratio of $O(\log k)$; that is, the within-cluster variation obtained by k -means++ is not more than $O(\log k)$ times larger than the global optimum.

2. Partitioning Methods

2.2 Variations of k -Means

Estimating the Number of Clusters

- The necessity for users to specify k , the number of clusters, in advance can be seen as a disadvantage.
- The desired number of k is often dependent on the shape and scale of the distribution of points in a data set and the desired clustering resolution of the user.
- Given a data set of n objects, let $B(k)$ and $W(k)$ be the sum of squares of the distances between and within clusters, respectively, when there are k clusters. The *Calinski-Harabasz index* is defined by

$$CH(k) = \frac{\frac{B(k)}{k-1}}{\frac{W(k)}{n-k}}$$

- The number of clusters k can be estimated by maximizing the Calinski-Harabasz index.

2. Partitioning Methods

2.2 Variations of k -Means

Estimating the Number of Clusters

- Gap statistic is another method to estimate the number of clusters. The sum of the pairwise distances for all points in a cluster C_i is

$$SC_{C_i} = \sum_{p, q \in C_i} \text{dist}(\mathbf{p}, \mathbf{q})$$

- If the data set is divided into k clusters, define

$$W_k = \sum_{i=1}^k \frac{SC_{C_i}}{2|C_i|}$$

which is the pooled within-cluster sum of squares around the cluster means. The gap statistic is

$$\text{Gap}_n(k) = E_n^*\{\log(W_k)\} - \log(W_k)$$

where E_n^* is the expectation under a sample of size n from the reference distribution, that is, the distribution producing the data set to be clustered. We can choose the value k that maximizes the gap statistic as the estimation of number of clusters.

2. Partitioning Methods

2.2 Variations of k -Means

Applying Feature Transformation

- The k -means method employing the Euclidean distance or any metric measures in general can only output convex clusters.
- Here, a cluster is convex if for any two points a and b belonging to the cluster, every point between the two points on the line connecting a and b also belongs to the cluster.
- Moreover, the k -means method employing any metric measure can **only** detect clusters that are linearly separable. That is, two clusters can be separated by a linear hyperplane.

2. Partitioning Methods

2.2 Variations of k -Means

Applying Feature Transformation

- In many applications, clusters may not be convex or linearly separable.

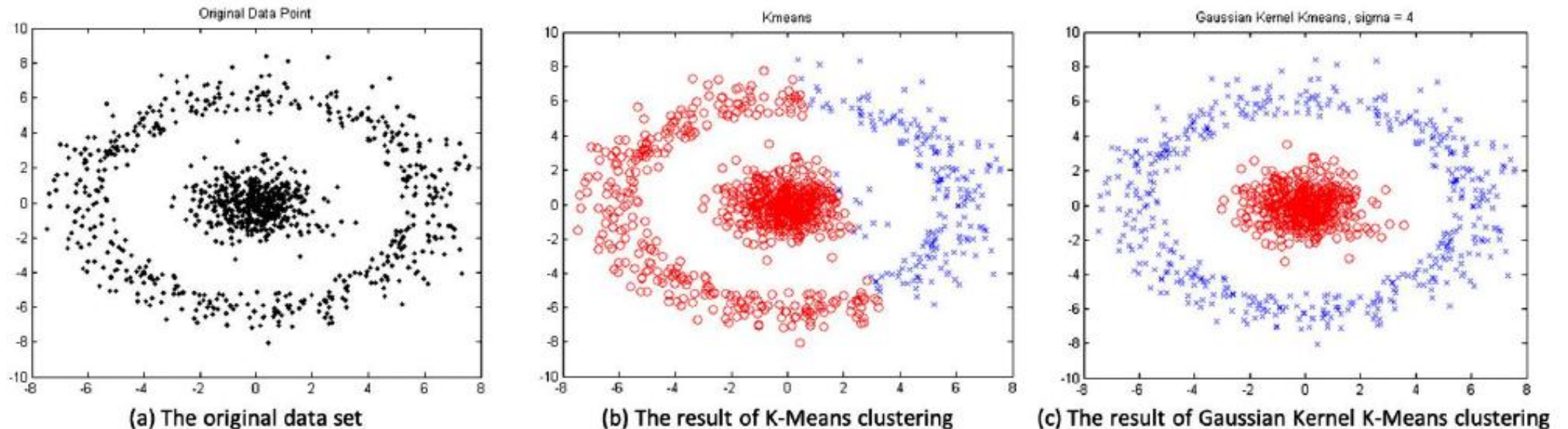


FIGURE 8.5

Concave and not linearly separable clusters can be detected by kernel k -means.

2. Partitioning Methods

2.2 Variations of k -Means

Applying Feature Transformation

- Can we still use the k -means method to find clusters that are concave and not linearly separable?
- Indeed, **kernel k -means** is such a method. The general idea of kernel k -means is to map the data points in the original input space to a feature space of higher dimensionality where the points belonging to the same cluster are close to each other in the feature space.
- Explicitly defining a space of high dimensionality and mapping the points into that space is subtle and may be costly. Instead, a convenient way is to apply a kernel function to measure the distance between points.

2. Partitioning Methods

2.2 Variations of k -Means

Applying Feature Transformation

- **Gaussian radial basis function (RBF) kernel**, we can calculate the distance between two points $\mathbf{x}$ and $\mathbf{y}$ by

$$K(\mathbf{x}, \mathbf{y}) = e^{-\frac{\|\mathbf{x} - \mathbf{y}\|^2}{2\sigma^2}}$$

- where $\|\mathbf{x} - \mathbf{y}\|^2$ is indeed the squared Euclidean distance between the two points, and σ is a free parameter.
- Clearly, the RBF kernel has the range between 0 and 1 and decreases with respect to the Euclidean distance.
- For example, if we apply the RBF kernel and k -means on the data set in Fig. 8.5(a), the output is Fig. 8.5(c), where the points in red form a cluster and the points in blue form another cluster.

2. Partitioning Methods

2.2 Variations of k -Means

Applying Feature Transformation

- Consider the five points in Fig. 8. The Euclidean distance matrix is

$$\begin{bmatrix} 0 & 5.66 & 5.66 & 5.66 & 5.66 \\ 5.66 & 0 & 8 & 11.31 & 8 \\ 5.66 & 8 & 0 & 8 & 11.31 \\ 5.66 & 11.31 & 8 & 0 & 8 \\ 5.66 & 8 & 11.31 & 8 & 0 \end{bmatrix}$$

where the value at the i th row and the j th column is the Euclidean distance between $\mathbf{x}_i$ and $\mathbf{x}_j$. Let $\sigma = 4$.

- The corresponding RBF kernel similarity matrix is

$$\begin{bmatrix} 1 & 0.37 & 0.37 & 0.37 & 0.37 \\ 0.37 & 1 & 0.135 & 0.02 & 0.135 \\ 0.37 & 0.135 & 1 & 0.135 & 0.02 \\ 0.37 & 0.02 & 0.135 & 1 & 0.135 \\ 0.37 & 0.135 & 0.02 & 0.135 & 1 \end{bmatrix}$$

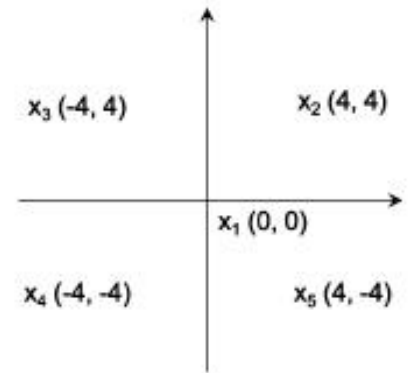


FIGURE 8.6

An example of five points.

3. Hierarchical Methods

3.1 Basic Concepts of Hierarchical Clustering

- Representing data objects in the form of a hierarchy is useful for data summarization and visualization.
- Consider handwritten character recognition.
 - A set of handwriting samples may be first partitioned into general groups where each group corresponds to a unique character. Some groups can be further partitioned into subgroups since a character may be written in multiple substantially different ways.

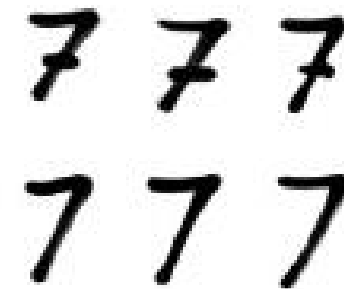


FIGURE 8.7

Two clusters of handwritten digit 7s.



3. Hierarchical Methods

3.1 Basic Concepts of Hierarchical Clustering

- A hierarchical clustering method can be either *agglomerative* or *divisive*, depending on whether the hierarchical decomposition is formed in a bottom-up (merging) or top-down (splitting) fashion.
- **Agglomerative hierarchical clustering method**
 - Uses a bottom-up strategy.
 - It typically starts by letting each object form its own cluster and iteratively merges clusters into larger and larger clusters, until all the objects are in a single cluster or certain termination conditions are satisfied. The single cluster becomes the hierarchy's root.
 - For the merging step, **it finds the two clusters that are closest to each other (according to some similarity measure) and combines the two to form one cluster.**
 - Because two clusters are merged per iteration, where each cluster contains at least one object, an agglomerative method requires at most n iterations.



3. Hierarchical Methods

3.1 Basic Concepts of Hierarchical Clustering

- **Divisive hierarchical clustering method**
 - Employs a top-down strategy.
 - It starts by placing all objects in one cluster, which is the hierarchy's root. It then divides the root cluster into several smaller subclusters and recursively partitions those clusters into smaller ones.
 - The partitioning process continues until each cluster at the lowest level is coherent enough—either containing only one object, or the objects within a cluster are sufficiently similar to each other.
- In either agglomerative or divisive hierarchical clustering, a user can specify the desired number of clusters as a termination condition.

3. Hierarchical Methods

3.1 Basic Concepts of Hierarchical Clustering

- Example 8.3. Agglomerative vs. divisive hierarchical clustering.** Fig. 8.8 shows the application of an agglomerative hierarchical clustering method and a divisive hierarchical clustering method on a data set of five objects, $\{a, b, c, d, e\}$.

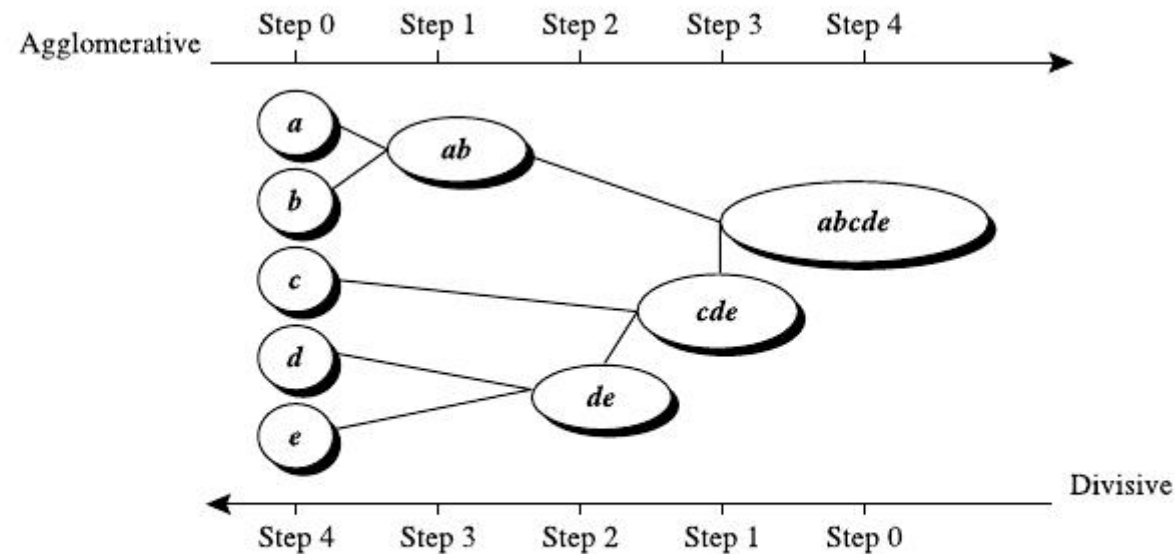


FIGURE 8.8

Agglomerative and divisive hierarchical clustering on data objects $\{a, b, c, d, e\}$.



3. Hierarchical Methods

3.1 Basic Concepts of Hierarchical Clustering

- Initially, the agglomerative method places each object into a cluster of its own.
- The clusters are then merged step-by-step according to some criterion.
 - For example, clusters C_1 and C_2 may be merged if an object in C_1 and an object in C_2 form the minimum Euclidean distance between any two objects from different clusters. This is a **single-linkage** approach in that each cluster is represented by all the objects in the cluster, and the similarity between two clusters is measured by the similarity of the **closest pair** of data points belonging to different clusters.
- The cluster-merging process repeats until all the objects are eventually merged to form one cluster.
- The divisive method proceeds in the contrasting way.
 - All the objects are used to form one initial cluster.
 - The cluster is split according to some principle such as **the maximum Euclidean distance between the closest neighboring objects in the cluster**.
 - The cluster-splitting process repeats until, eventually, each new cluster contains only a single object.

3. Hierarchical Methods

3.2 Agglomerative Hierarchical Clustering

Similarity Measures in Hierarchical Clustering

- *How can we choose which objects and clusters to merge in an agglomerative step?*
- The core is to measure the similarity between two clusters, where each cluster is generally a set of objects.
- Four widely used measures for distance between clusters are as follows, where $\|\mathbf{p} - \mathbf{p}'\|$ is the distance between two objects or points, $\mathbf{p}$ and $\mathbf{p}'$; $\mathbf{m}_i$ is the mean for cluster C_i ; and n_i is the number of objects in C_i . They are also known as *linkage measures*.

3. Hierarchical Methods

3.2 Agglomerative Hierarchical Clustering

Similarity Measures in Hierarchical Clustering

- Measures for distance between clusters (linkage measures)
 - Minimum distance: $dist_{min}(C_i, C_j) = \min_{\mathbf{p} \in C_i, \mathbf{p}' \in C_j} \{||\mathbf{p} - \mathbf{p}'||\}$
 - Maximum distance: $dist_{max}(C_i, C_j) = \max_{\mathbf{p} \in C_i, \mathbf{p}' \in C_j} \{||\mathbf{p} - \mathbf{p}'||\}$
 - Mean distance: $dist_{mean}(C_i, C_j) = ||\mathbf{m}_i - \mathbf{m}_j||$
 - Average distance: $dist_{average}(C_i, C_j) = \frac{1}{n_i n_j} \sum_{\mathbf{p} \in C_i, \mathbf{p}' \in C_j} ||\mathbf{p} - \mathbf{p}'||$

3. Hierarchical Methods

3.2 Agglomerative Hierarchical Clustering

Similarity Measures in Hierarchical Clustering

- When an algorithm uses the *minimum distance*, $dist_{min}(C_i, C_j)$, to measure the distance between clusters:
 - It is sometimes called a **nearest-neighbor clustering algorithm** or **single-linkage algorithm**.
 - If we view the data points as nodes of a graph, with edges forming a path between the nodes in a cluster, then the merging of two clusters, C_i and C_j , corresponds to adding an edge between the nearest pair of nodes in C_i and C_j .
- When an algorithm uses the *maximum distance*, $dist_{max}(C_i, C_j)$, to measure the distance between clusters:
 - It is sometimes called a **farthest-neighbor clustering algorithm** or **complete-linkage algorithm**.
 - By viewing data points as nodes of a graph, with edges linking nodes, we can think of each cluster as a complete subgraph, that is, with edges connecting all the nodes in the clusters. The distance between two clusters is determined by the most distant nodes in the two clusters.

3. Hierarchical Methods

3.2 Agglomerative Hierarchical Clustering

Similarity Measures in Hierarchical Clustering

- Similar to the situation in the k -means method, single-linkage and complete-linkage methods are sensitive to outliers.
- The use of *mean* or *average distance* is a compromise between the minimum and maximum distances and overcomes the outlier sensitivity problem.
- Whereas the *mean distance* is the simplest to compute, the *average distance* is advantageous in that it can handle categorical data and numeric data. The computation of the mean vector for categorical data can be difficult or impossible to define.

3. Hierarchical Methods

3.2 Agglomerative Hierarchical Clustering

Similarity Measures in Hierarchical Clustering

- Example 8.4. Single vs. complete linkages.** Let us apply hierarchical clustering to the data set of Fig. 8.9(a). Fig. 8.9(b) shows the hierarchy of clusters using single-linkage. Fig. 8.9(c) shows the case using complete linkage, where the edges between clusters $\{A, B, J, H\}$ and $\{C, D, G, F, E\}$ are omitted for ease of presentation.

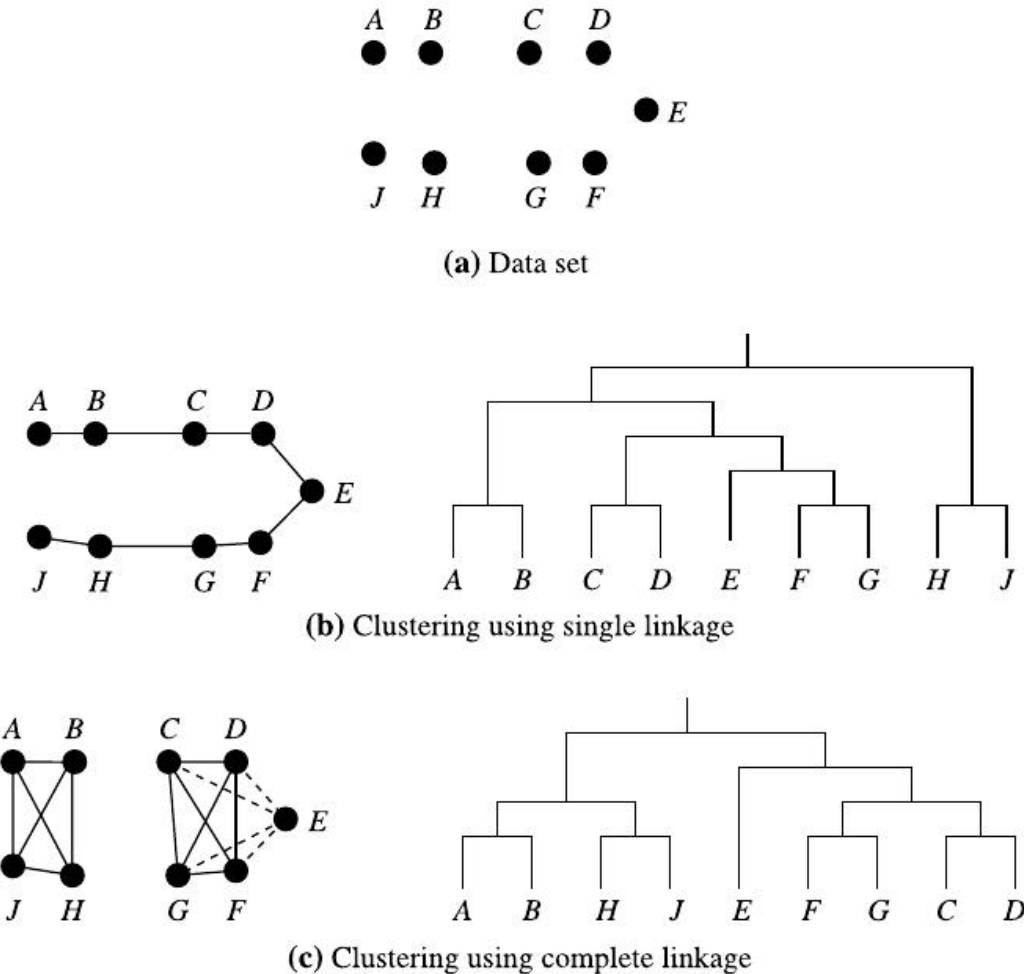


FIGURE 8.9

Hierarchical clustering using single and complete linkages.

3.2 Agglomerative Hierarchical Clustering

Connecting Agglomerative Hierarchical Clustering and Partitioning Methods

- *Are there any connections between (agglomerative) hierarchical clustering and partitioning methods?*
- Partitioning methods use the sum of squared errors (SSE) to measure the compactness and quality of a possible clustering, that is a partitioning of points into clusters.
- Heuristically, agglomerative hierarchical clustering methods may also use the sum of squared errors (SSE) to guide the selection of clusters to merge.

3.2 Agglomerative Hierarchical Clustering

Connecting Agglomerative Hierarchical Clustering and Partitioning Methods

- For a data set of n points, if we set the number of clusters to n , a partitioning method naturally assigns each point into a cluster. This corresponds to the starting point of agglomerative hierarchical clustering.
- When we merge clusters in agglomerative clustering, we reduce the number of clusters.
 - Which two clusters should we choose to merge? Heuristically, we may want to merge two clusters so that the resulting clustering also minimizes the sum of squared errors (SSE), which is used as the criterion in partitioning methods like k -means.

3.2 Agglomerative Hierarchical Clustering

Connecting Agglomerative Hierarchical Clustering and Partitioning Methods

- Consider the five clusters in Fig. 8.10 as an illustrative example. Suppose we want to merge two of them into one so that we can have a hierarchy of clusters. Among all the possible pairs of clusters, merging C_1 and C_2 minimizes the SSE, and thus C_1 and C_2 should be merged in the next step in building the hierarchy.

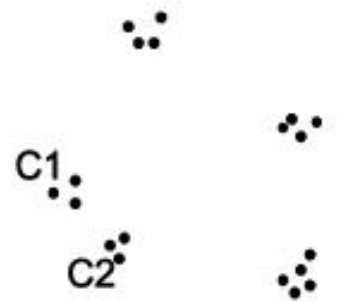


FIGURE 8.10

Ward's criterion.

- The above intuition connecting agglomerative hierarchical clustering and partitioning methods gives us an alternative way to measure the similarity between two clusters. **We can look at the increase of the SSE if the two clusters are merged into one, the smaller the better.** This is formulated by J.H. Ward and thus is known as **Ward's criterion**.

3.2 Agglomerative Hierarchical Clustering

Connecting Agglomerative Hierarchical Clustering and Partitioning Methods

- Suppose two disjoint clusters C_i and C_j are merged, and $m_{(ij)}$ is the mean of the new cluster. Then, Ward's criterion is defined as

$$\begin{aligned} W(C_i, C_j) &= \sum_{\mathbf{x} \in C_i \cup C_j} \|\mathbf{x} - \mathbf{m}_{(ij)}\|^2 - \sum_{\mathbf{x} \in C_i} \|\mathbf{x} - \mathbf{m}_i\|^2 - \sum_{\mathbf{x} \in C_j} \|\mathbf{x} - \mathbf{m}_j\|^2 \\ &= \frac{n_i n_j}{n_i + n_j} \|\mathbf{m}_i - \mathbf{m}_j\|^2 \end{aligned}$$

3. Hierarchical Methods

3.2 Agglomerative Hierarchical Clustering

The Lance-William Algorithm

- *We discuss a few different proximity measures for clusters in agglomerative hierarchical clustering, is there a way to generalize them?* Indeed, the **Lance-Williams formula** generalizes different measures in a uniform way.

- Suppose two exclusive clusters C_i and C_j are merged. We need to specify the distance between the merged cluster, denoted by $C_{(ij)}$, and every other cluster C_k . The similarity between the merged clusters $C_{(ij)}$ and cluster C_k is given by

$$d(C_{(ij)}, C_k) = \alpha_i d(C_i, C_k) + \alpha_j d(C_j, C_k) + \beta d(C_i, C_j) + \gamma |d(C_i, C_k) - d(C_j, C_k)|$$

where α_i , α_j , β , and γ are parameters that together with the similarity function $d(C_i, C_j)$ determine the hierarchical clustering algorithm.

- The single-linkage method is equivalent to taking $\alpha_i = \alpha_j = 0.5$, $\beta = 0$, and $\gamma = -0.5$.
- The complete-linkage method is equivalent to $\alpha_i = \alpha_j = 0.5$, $\beta = 0$, and $\gamma = 0.5$.
- To implement the Ward's criterion, we can take $\alpha_i = \frac{n_i + n_k}{n_i + n_j + n_k}$, $\alpha_j = \frac{n_j + n_k}{n_i + n_j + n_k}$, $\beta = -\frac{n_k}{n_i + n_j + n_k}$ and $\gamma = 0$ in the Lance-William formula.

3. Hierarchical Methods

3.2 Agglomerative Hierarchical Clustering

The Lance-William Algorithm

- Using the Lance-Williams formula, the *Lance-Williams algorithm* generalizes agglomerative hierarchical clustering.
- It takes an agglomerative way and minimizes the sum of distance in each iteration until all points are merged into one cluster.



3. Hierarchical Methods

3.3 Divisive Hierarchical Clustering

- A divisive hierarchical clustering method partitions a set of objects step by step into clusters. To design a divisive hierarchical clustering method, there are three important issues to consider.
 - 1. A set of objects can be split in many different ways. A splitting criterion is needed to determine which splitting is the best. Technically, given two splittings, the splitting criterion should be able to tell which one is better.
 - For example, the SSE may be used on numeric data. If two splittings have the same number of clusters, then the one with a less SSE value is preferred. On nominal data, the Gini index (Chapter 6) may be used.
 - 2. While in general the splitting method should optimize the splitting criterion, one major consideration is the computational cost. For example, enumerating all possible splittings and finding the best one is likely computationally prohibitive. Thus some heuristic or approximate methods, such as bisecting k -means, that is, setting $k = 2$, may be used.
 - 3. In the middle of divisive hierarchical clustering, there are in general multiple clusters. Then, which cluster should be split next? An intuitive idea is to choose the “loosest” cluster. More concretely, we may compute the average SSE of each cluster, $E_{C_i} = \frac{1}{|C_i|} \sum_{\mathbf{x} \in C_i} (\mathbf{x} - \mathbf{m}_i)^2$, and choose the one with the largest average SSE to split.

3. Hierarchical Methods

3.3 Divisive Hierarchical Clustering

The Minimum Spanning Tree-Based Approach

- In a weighted graph G , a minimum spanning tree is an acyclic subgraph that contains all nodes in G , and the sum of edge weights of the tree is minimized.
 - Minimum spanning tree can be computed using, for example, Prim's algorithm and Kruskal's algorithm.

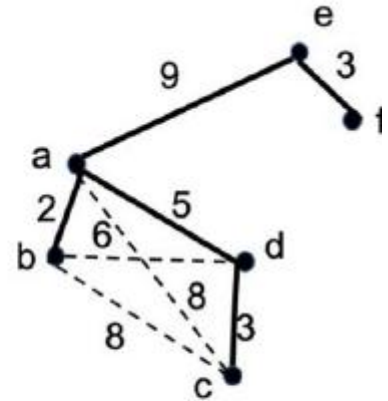


FIGURE 8.11

A weighted graph and a minimum spanning tree.

3. Hierarchical Methods

3.3 Divisive Hierarchical Clustering

The Minimum Spanning Tree-Based Approach

- Given a set of points, we can construct a weighted graph such that each point is represented by a node in the graph, and the distance between two points is the weight of the edge connecting the two corresponding nodes in the graph.
- Then, we can compute the minimum spanning tree of the weighted graph.
- Intuitively, the minimum spanning tree can be regarded as the most compact way that the points are connected into one cluster.
- In general, in the process of divisive hierarchical clustering based on minimum spanning tree, every cluster is a subset of nodes and is represented by the minimum spanning tree, which is a subtree of the minimum spanning tree of the whole data set.
- **The splitting criterion is the total weights of all the edges in the spanning tree(s) of all the clusters, the smaller the better.**

3. Hierarchical Methods

3.3 Divisive Hierarchical Clustering

The Minimum Spanning Tree-Based Approach

- Based on this spanning tree, we can progressively divide the set of points in one cluster into smaller clusters.
- Suppose we want to conduct bisecting splitting, that is, every time we split one cluster into two smaller clusters.
- At each step, we consider all edges in the spanning trees of the current clusters and **delete the edge of the largest weight**.
- Deleting an edge in a tree divides one cluster into two. Thus the splitting method is to divide a cluster by deleting the edge in the minimum spanning tree of the largest weight.

3. Hierarchical Methods

3.3 Divisive Hierarchical Clustering

The Minimum Spanning Tree-Based Approach

- For example, consider a set of points $\{a, b, c, d, e, f\}$ as shown in Fig. 8.11. Based on the minimum spanning tree method, the edge (a, e) , which has the largest weight in the minimum spanning tree, is first deleted, which divides the data set into two clusters, $\{a, b, c, d\}$ and $\{e, f\}$. Next, by deleting edge (a, d) , which has the largest weight in the remaining minimum spanning trees, the cluster $\{a, b, c, d\}$ is split into two smaller clusters $\{a, b\}$ and $\{c, d\}$. The process continues until each cluster has only one point and all the edges in the minimum spanning tree are deleted.

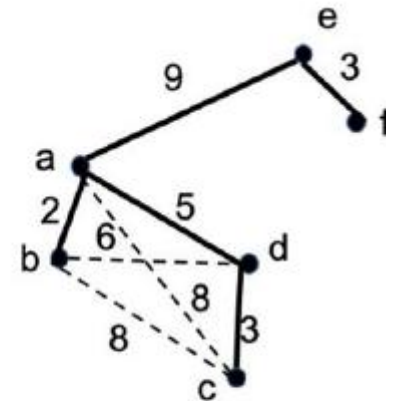


FIGURE 8.11

A weighted graph and a minimum spanning tree.

3. Hierarchical Methods

3.3 Divisive Hierarchical Clustering

Dendrogram

- A tree structure called a **dendrogram** is commonly used to represent the process of hierarchical clustering.
- It shows how objects are grouped together (in an agglomerative method) or partitioned (in a divisive method) step-by-step.

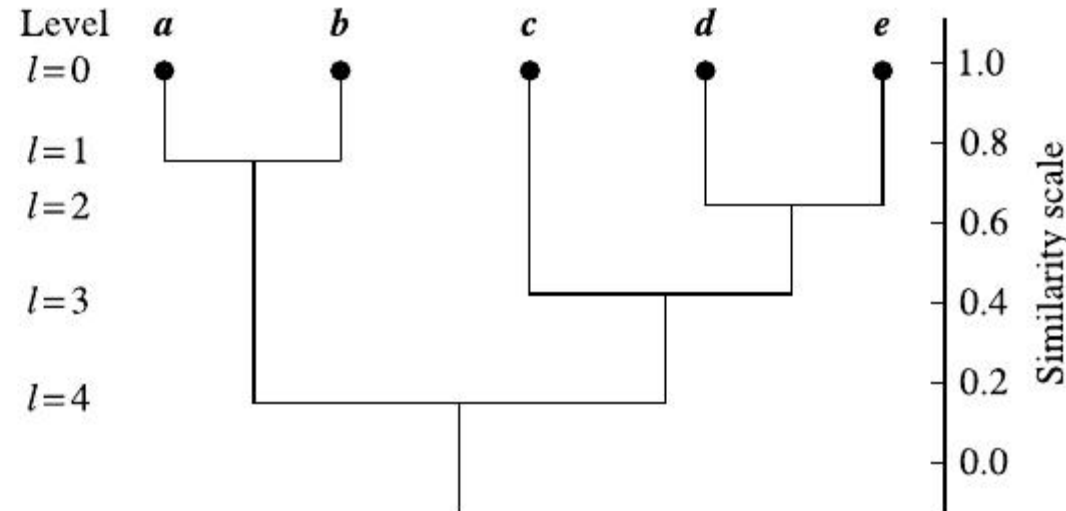


FIGURE 8.12

Dendrogram representation for hierarchical clustering of data objects $\{a, b, c, d, e\}$.



3. Hierarchical Methods

3.4 Probabilistic Hierarchical Clustering

- **Probabilistic hierarchical clustering** uses probabilistic models to measure distances between clusters.
- One way to look at the clustering problem is to regard the set of data objects to be clustered as a sample of the underlying data generation mechanism to be analyzed or, formally, the *generative model*.
- The task of clustering is to estimate the generative model as accurately as possible using the observed data objects to be clustered.
- In practice, we can assume that the data generative models adopt common distribution functions, such as Gaussian distribution or Bernoulli distribution, which are governed by parameters. The task of learning a generative model is then reduced to finding the parameter values for which the model best fits the observed data set.

3. Hierarchical Methods

3.4 Probabilistic Hierarchical Clustering

- **Example 8.6. Generative model.** Suppose we are given a set of 1-D points $X = \{x_1, \dots, x_n\}$ for clustering analysis. Let us assume that the data points are generated by a Gaussian distribution,

$$\mathcal{N}(\mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}},$$

where the parameters are μ (the mean) and σ^2 (the variance).

- The probability that a point $x_i \in X$ is then generated by the model is

$$P(x_i | \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x_i - \mu)^2}{2\sigma^2}}.$$

- Consequently, the likelihood that the data set X observed is generated by the model is

$$L(\mathcal{N}(\mu, \sigma^2) : X) = P(X | \mu, \sigma^2) = \prod_{i=1}^n \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x_i - \mu)^2}{2\sigma^2}}.$$

3. Hierarchical Methods

3.4 Probabilistic Hierarchical Clustering

- The task of learning the generative model is to find the parameters μ and σ^2 such that the likelihood $L(N(\mu, \sigma^2):X)$ is maximized, that is, finding

$$\mathcal{N}(\mu_0, \sigma_0^2) = \arg \max \{L(\mathcal{N}(\mu, \sigma^2) : X)\},$$

where $\max \{L(N(\mu, \sigma^2):X)\}$ is called the *maximum likelihood*.

- Given a set of objects, the quality of a cluster formed by all the objects can be measured by the maximum likelihood. For a set of objects partitioned into m clusters $C_1, \dots, C_m$. Then the quality can be measured by

$$Q(\{C_1, \dots, C_m\}) = \prod_{i=1}^m P(C_i),$$

where $P()$ is the maximum likelihood.

3. Hierarchical Methods

3.4 Probabilistic Hierarchical Clustering

- To calculate $P(C_i)$, we can fit each cluster C_i ($1 \leq i \leq m$) by a generative model M_i , and estimate the probability by

$$P(C_i) = \prod_{x \in C_i} P(x|M_i)$$

- If we merge two clusters, C_{j_1} and C_{j_2} , into a cluster, $C_{j_1} \cup C_{j_2}$, then, the change in quality of the overall clustering is
- Merging two clusters may **not** always lead to an improvement in clustering quality, that is,

$$\begin{aligned} & Q((\{C_1, \dots, C_m\} - \{C_{j_1}, C_{j_2}\}) \cup \{C_{j_1} \cup C_{j_2}\}) - Q(\{C_1, \dots, C_m\}) \\ &= \frac{\prod_{i=1}^m P(C_i) \cdot P(C_{j_1} \cup C_{j_2})}{P(C_{j_1})P(C_{j_2})} - \prod_{i=1}^m P(C_i) \\ &= \prod_{i=1}^m P(C_i) \left(\frac{P(C_{j_1} \cup C_{j_2})}{P(C_{j_1})P(C_{j_2})} - 1 \right). \end{aligned}$$

$$\frac{P(C_{j_1} \cup C_{j_2})}{P(C_{j_1})P(C_{j_2})}$$

may be less than 1.

Merging clusters two clusters may lower the clustering quality because no Gaussian functions can fit the merged cluster well.



3. Hierarchical Methods

3.4 Probabilistic Hierarchical Clustering

- Given clusters C_1 and C_2 , the dissimilarity between them can be measured by

$$\text{dist}(C_1, C_2) = -\log \frac{P(C_1 \cup C_2)}{P(C_1)P(C_2)}.$$

- A probabilistic hierarchical clustering scheme can start with one cluster per object and merge two clusters, C_i and C_j , if the distance between them is **negative**.
- In each iteration, we try to find C_i and C_j so as to maximize $\log \frac{P(C_i \cup C_j)}{P(C_i)P(C_j)}$.
- The iteration continues as long as $\log \frac{P(C_i \cup C_j)}{P(C_i)P(C_j)} > 0$, that is, as long as there is an improvement in clustering quality.

3. Hierarchical Methods

3.4 Probabilistic Hierarchical Clustering

Algorithm: A probabilistic hierarchical clustering algorithm.

Input:

- $D = \{o_1, \dots, o_n\}$: a data set containing n objects;

Output: A hierarchy of clusters.

Method:

- (1) **create** a cluster for each object $C_i = \{o_i\}$, $1 \leq i \leq n$;
- (2) **for** $i = 1$ to n
- (3) **find** pair of clusters C_i and C_j such that $C_i, C_j = \arg \max_{i \neq j} \log \frac{P(C_i \cup C_j)}{P(C_i)P(C_j)}$;
- (4) **if** $\log \frac{P(C_i \cup C_j)}{P(C_i)P(C_j)} > 0$ **then** merge C_i and C_j ;
- (5) **else** stop;

FIGURE 8.15

A probabilistic hierarchical clustering algorithm.

4. Density-Based and Grid-Based Methods

- Most of the partitioning and hierarchical methods are designed to find spherical-shaped clusters. They have difficulty finding clusters of arbitrary shape such as the “S” shape and oval clusters in Fig. 8.16.
- Although some feature transformation methods, such as kernel k -means, may help, it is often tricky to choose appropriate kernel functions. Given such data, they would likely inaccurately identify convex regions, where noises or outliers are included in the clusters.
- To find clusters of arbitrary shape, alternatively, we can model clusters as dense regions in the data space, separated by sparse regions. This is the main strategy behind *density-based clustering methods*, which can discover clusters of nonspherical shape.

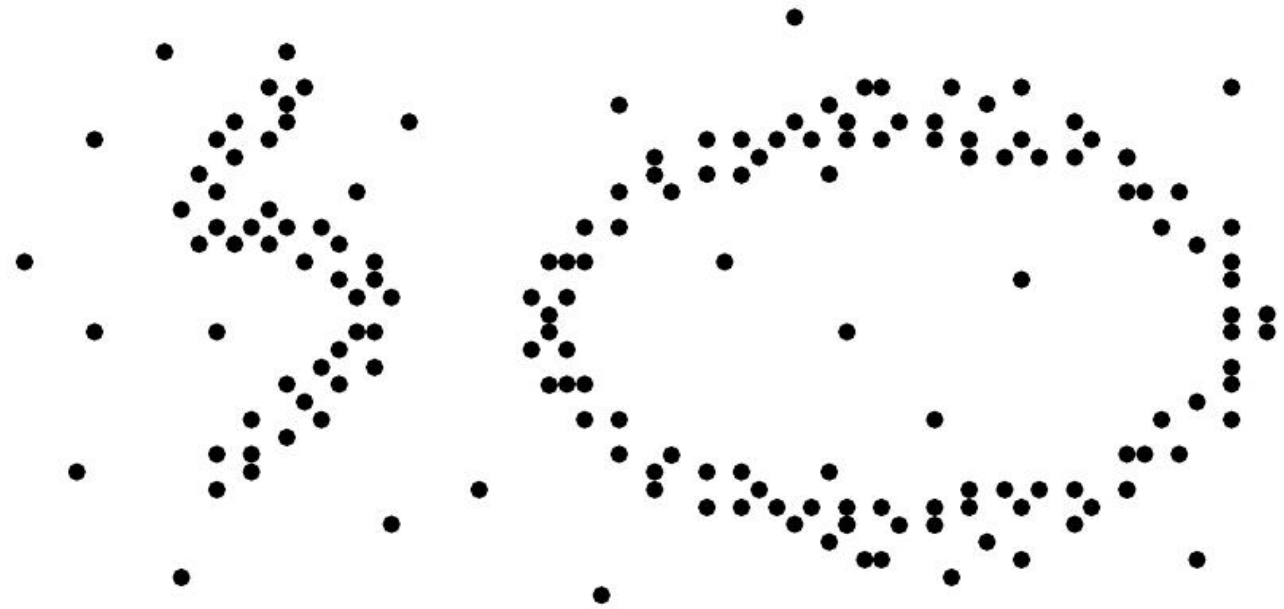


FIGURE 8.16

Clusters of arbitrary shape.



4. Density-Based and Grid-Based Methods

4.1 DBSCAN

- “*How can we find dense regions in density-based clustering?*”
- The density of an object o can be measured by the number of objects close to o .
- **DBSCAN** (Density-Based Spatial Clustering of Applications with Noise) finds *core objects*, that is, objects that have dense neighborhoods.
 - It connects core objects and their neighborhoods to form dense regions as clusters.
- **DBSCAN***: an improved version of the original **DBSCAN**.



4. Density-Based and Grid-Based Methods

4.1 DBSCAN

- “How does **DBSCAN*** quantify the neighborhood of an object?”
- DBSCAN* employs a user specified parameter $\epsilon > 0$ to specify the radius of a neighborhood.
- The **ϵ -neighborhood** of an object o is the space within a radius ϵ centered at o .
- Due to the fixed neighborhood size parameterized by ϵ , the density of a neighborhood can be measured simply by the number of objects in the neighborhood.
- To determine whether a neighborhood is dense or not, DBSCAN* uses another user-specified parameter, *MinPts*, which specifies the density threshold of dense regions.
- An object is a **core object** if the ϵ -neighborhood of the object contains at least *MinPts* objects, otherwise, it is **noise**. Core objects are the pillars of dense regions.
- Given a set, D , of objects, we can identify all core objects with respect to the given parameters, ϵ and *MinPts*. The clustering task is therein reduced to using core objects and their neighborhoods to form dense regions, where the dense regions are clusters.



4. Density-Based and Grid-Based Methods

4.1 DBSCAN

- Two core objects p and q are **ϵ -reachable** if $d(p, q) \leq \epsilon$, that is, p is in the ϵ -neighbor of q and vice versa.
- Two core objects p and q are **density-connected** if p and q are ϵ -reachable or transitively ϵ -reachable, where p and q are transitively ϵ -reachable if there exist one or multiple core objects $r_1, \dots, r_l$ such that p and r_1 are ϵ -reachable, r_i and r_{i+1} ($1 \leq i < l$) are ϵ -reachable, and r_l and q are ϵ -reachable.
- Then, a cluster C with respect to parameters ϵ and $MinPts$ is simply a nonempty maximal subset of core objects that every pair of objects in C is density connected.
- In DBSCAN*, a cluster contains only core objects. However, it is easy to assign a noncore object that is in the ϵ -neighborhood of a core object to the cluster containing that core object.
- DBSCAN explicitly identifies such noncore objects as border objects, and DBSCAN* can use a postprocessing step to pick up those border objects. Those objects that do not belong to any ϵ -neighborhood are outliers.

4. Density-Based and Grid-Based Methods

4.1 DBSCAN

- **Example 8.7. Density-reachability and density-connectivity.** Consider Fig. 8.17 for a given ϵ represented by the radius of the circles, and, say, let $MinPts = 3$.
- p, m, o, q, t : core objects
- p and o , o and q : ϵ -reachable.
- p and q : density-connected.
- p, m, o, q, t : a cluster
- s is not a core object, since the ϵ -neighborhood of s contains only two objects.
- However, s is in the ϵ -neighborhood of core object p , thus s is a border object.
- u and v are not core objects, and they do not belong to the ϵ -neighborhood of any core objects. Thus they are outliers.

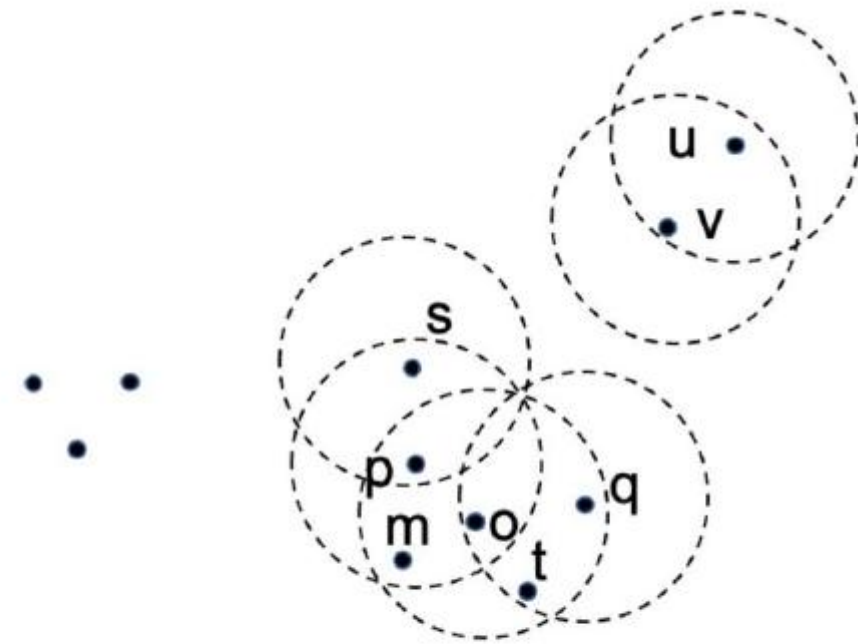


FIGURE 8.17

Density-reachability and density-connectivity in DBSCAN.

4. Density-Based and Grid-Based Methods

4.1 DBSCAN

Algorithm: DBSCAN*: a density-based clustering algorithm.

Input:

- D : a data set containing n objects,
- ϵ : the radius parameter, and
- $MinPts$: the neighborhood density threshold.

Output: A set of density-based clusters.

Method:

```
(1)  mark all objects as unvisited;
(2)  do
(3)      randomly select an unvisited object  $p$ ;
(4)      mark  $p$  as visited;
(5)      if the  $\epsilon$ -neighborhood of  $p$  has at least  $MinPts$  objects
(6)          create a new cluster  $C$ , and add  $p$  to  $C$ ;
(7)          let  $N$  be the set of objects in the  $\epsilon$ -neighborhood of  $p$ ;
(8)          for each point  $p'$  in  $N$ 
(9)              if  $p'$  is unvisited
(10)                  mark  $p'$  as visited;
(11)                  if the  $\epsilon$ -neighborhood of  $p'$  has at least  $MinPts$  points,
                      add those points to  $N$  and add  $p'$  to  $C$ ;
(12)          end for
(13)          output  $C$ ;
(14)      else mark  $p$  as noise;
(15) until no object is unvisited;
```

FIGURE 8.18

DBSCAN* algorithm.



4. Density-Based and Grid-Based Methods

4.2 DENCLUE

- **DENCLUE (DENSity-based CLUstEring)** is a clustering method based on a set of density distribution functions.
- **Density estimation** is a core issue in density-based clustering.
 - In probability and statistics, **density estimation** is the estimation of an unobservable underlying probability density function based on a set of observed data.
 - In the context of density-based clustering, the unobservable underlying probability density function is the true distribution of the population of all possible objects to be analyzed. The observed data set is regarded as a random sample from that population.
- In DENCLUE, **kernel density estimation** is used, which is a nonparametric density estimation approach from statistics.
 - We treat an observed object as an indicator of high-probability density in the surrounding region. The probability density at a point depends on the distances from this point to the observed objects.

4. Density-Based and Grid-Based Methods

4.2 DENCLUE

- Let $\mathbf{x}_1, \dots, \mathbf{x}_n$ be an independent and identically distributed sample of a random variable f . The *kernel density approximation of the probability density function* is

$$\hat{f}_h(\mathbf{x}) = \frac{1}{nh} \sum_{i=1}^n K\left(\frac{\mathbf{x} - \mathbf{x}_i}{h}\right)$$

where $K()$ is a kernel function and h is the bandwidth serving as a smoothing parameter.

- A **kernel** can be regarded as a function modeling the influence of a sample point within its neighborhood. Technically, a kernel $K()$ is a nonnegative real-valued integrable function that should satisfy two requirements:

- $\int_{-\infty}^{+\infty} K(u) du = 1$
- $K(-u) = K(u)$ for all values of u .

- A frequently used kernel is a standard Gaussian function with a mean of 0 and a variance of 1:

$$K\left(\frac{\mathbf{x} - \mathbf{x}_i}{h}\right) = \frac{1}{\sqrt{2\pi}} e^{-\frac{(\mathbf{x} - \mathbf{x}_i)^2}{2h^2}}$$



4. Density-Based and Grid-Based Methods

4.2 DENCLUE

- DENCLUE uses a Gaussian kernel to estimate density based on the given set of objects to be clustered.
- A point $\mathbf{x}^*$ is called a **density attractor** if it is a local maximum of the estimated density function.
- To avoid trivial local maximum points, DENCLUE uses a noise threshold, ξ , and only considers those density attractors $\mathbf{x}^*$ such that $\hat{f}_h(\mathbf{x}^*) \geq \xi$. These nontrivial density attractors are the centers of clusters.
- The objects under analysis are assigned to clusters through density attractors using a stepwise hill-climbing procedure.

4. Density-Based and Grid-Based Methods

4.2 DENCLUE

- For an object, $\mathbf{x}$, the hill-climbing procedure starts from $\mathbf{x}$ and is guided by the gradient of the estimated density function. That is, the density attractor for $\mathbf{x}$ is computed as

$$\begin{aligned} \mathbf{x}^0 &= \mathbf{x} \\ \mathbf{x}^{j+1} &= \mathbf{x}^j + \delta \frac{\nabla \hat{f}(\mathbf{x}^j)}{|\nabla \hat{f}(\mathbf{x}^j)|}, \end{aligned}$$

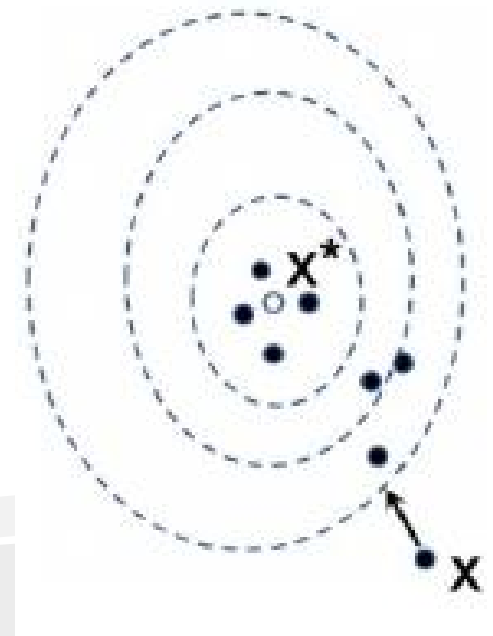
where δ is a parameter to control the speed of convergence, and

$$\nabla \hat{f}(\mathbf{x}) = \frac{1}{h^{d+2} n \sum_{i=1}^n K\left(\frac{\mathbf{x} - \mathbf{x}_i}{h}\right) (\mathbf{x}_i - \mathbf{x})}.$$

4. Density-Based and Grid-Based Methods

4.2 DENCLUE

- The hill-climbing procedure stops at step $k > 0$ if $\hat{f}(\mathbf{x}^{k+1}) < \hat{f}(\mathbf{x}^k)$, and assigns $\mathbf{x}$ to the density attractor $\mathbf{x}^* = \mathbf{x}^k$. An object $\mathbf{x}$ is an outlier or noise if it converges in the hill-climbing procedure to a local maximum $\mathbf{x}^*$ with $\hat{f}(\mathbf{x}^k) < \xi$.
- Hill-climbing idea:
 - For a point $\mathbf{x}$, the density attractor for $\mathbf{x}$ is initialized to $\mathbf{x}^0 = \mathbf{x}$. In the next iteration, the density attractor moves a small step towards the direction indicated by the gradient of the density function, shown by the arrow in the figure, until the point $\mathbf{x}^*$ where density is stable (the white circle point in the figure), which is a local optimal.





4. Density-Based and Grid-Based Methods

4.2 DENCLUE

- A cluster in DENCLUE is a set of density attractors X and a set of input objects C such that each object in C is assigned to a density attractor in X , and there exists a path between every pair of density attractors where the density is above ξ .
- By using multiple density attractors connected by paths, DENCLUE can find clusters of arbitrary shape.